

Cliff Walking 強化學習比較



Cliff Walking : Q-learning vs SARSA

一、作業目的

本作業旨在實作並比較兩種經典強化學習演算法：

* Q-learning (Off-policy)

* SARSA (On-policy)

透過相同環境與參數設定，分析兩者在學習過程中的：

* 收斂速度

* 策略行為

* 穩定性差異

二、環境描述 (Cliff Walking)

本實驗採用 Gridworld 環境 (4 × 12)：

* 起點 (Start)：左下角

* 終點 (Goal)：右下角

* 懸崖 (Cliff)：底部中間區域

獎勵機制：

* 每移動一步：**-1**

* 掉入懸崖：**-100**，並回到起點

* 到達終點：**0 (回合結束)**

三、方法說明

1 Q-learning (Off-policy)

更新公式：

$$[Q(s,a) \leftarrow Q(s,a) + \alpha (r + \gamma \max_a Q(s',a) - Q(s,a))]]$$

- * 使用「最佳下一步」來更新
- * 偏向學習**最優策略（貪婪）**
- * 可能較冒險（貼近懸崖）

2 SARSA (On-policy)

更新公式：

$$[Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma Q(s',a') - Q(s,a) \right]]$$

- * 使用「實際採取的下一步」
- * 考慮 exploration (ϵ -greedy)
- * 策略較保守

3 參數設定

參數	數值
-----	-----
α (學習率)	0.1
γ (折扣因子)	0.9
ϵ (探索率)	0.1
Episodes	500
Runs	50 (取平均)

四、實驗結果

1 學習曲線 (Learning Curve)

輸出圖：

`learning_curve.png`

觀察結果：

- * 初期 reward 非常低（約 -100），因為頻繁掉入 cliff
- * 隨著訓練進行，reward 快速上升
- * 最終：

* SARSA 約落在 ** -20 ~ -30 **

* Q-learning 約落在 ** -40 ~ -50 **

2 策略比較 (Policy)

輸出圖：

q_policy.png
sarsa_policy.png

Q-learning :

- * 學習到**最短路徑**
- * 會貼著 cliff 行走
- * 風險高 (容易掉落)

SARSA :

- * 學習到**較安全路徑**
- * 遠離 cliff
- * 雖然路徑較長，但更穩定

3 穩定性分析

方法	特性
-----	-----
Q-learning	波動較大、較激進
SARSA	較平穩、較保守

原因：

- * SARSA 把 ϵ -greedy 的隨機性納入學習
- * Q-learning 假設未來永遠選最佳動作 (過於理想)

五、重要觀察 (重點！老師很愛看這段)

1. **Q-learning 是理想化策略**

- * 學的是「最優解」
- * 但實際執行可能危險

2. **SARSA 是實際策略**

- * 學的是「會探索的策略」
- * 因此更安全

3. **Cliff Walking 是經典對比例子**

- * 完美展示 On-policy vs Off-policy 差異

六、結論

- * Q-learning 收斂較快，但策略較冒險
- * SARSA 收斂較穩定，策略較安全
- * 在高風險環境中：
 - 👉 SARSA 通常表現較好

七、如何執行

```
bash
python main.py
```

輸出：

- * learning_curve.png
- * q_policy.png
- * sarsa_policy.png

八、程式架構

- `CliffWalkingEnv`：環境
- `QLearningAgent`：Q-learning
- `SARSAgent`：SARSA
- `run\_experiment()`：多次平均
- `visualize\_policy()`：策略視覺化

```
import numpy as np
import random
import matplotlib.pyplot as plt
```

```
# =====
# Environment
# =====
class CliffWalkingEnv:
    def __init__(self, rows=4, cols=12):
```

```

self.rows = rows
self.cols = cols
self.start = (rows - 1, 0)
self.goal = (rows - 1, cols - 1)
self.reset()

def reset(self):
    self.state = self.start
    return self.state

def step(self, action):
    r, c = self.state

    # 0: Up, 1: Right, 2: Down, 3: Left
    if action == 0:
        r = max(r - 1, 0)
    elif action == 1:
        c = min(c + 1, self.cols - 1)
    elif action == 2:
        r = min(r + 1, self.rows - 1)
    elif action == 3:
        c = max(c - 1, 0)

    next_state = (r, c)

    # Cliff
    if r == self.rows - 1 and 1 <= c <= self.cols - 2:
        reward = -100
        next_state = self.start
        done = False
    elif next_state == self.goal:
        reward = 0
        done = True
    else:
        reward = -1
        done = False

    self.state = next_state
    return next_state, reward, done

# =====
# Base Agent
# =====
class BaseAgent:
    def __init__(self, rows, cols, actions,
                 alpha=0.1, gamma=0.9, epsilon=0.1):

        self.q = np.zeros((rows, cols, actions))
        self.alpha = alpha
        self.gamma = gamma

```

```

self.epsilon = epsilon
self.actions = actions

def choose_action(self, state):
    if random.random() < self.epsilon:
        return random.randint(0, self.actions - 1)
    else:
        r, c = state
        return np.argmax(self.q[r, c])

# =====
# Q-Learning
# =====
class QLearningAgent(BaseAgent):
    def update(self, state, action, reward, next_state):
        r, c = state
        nr, nc = next_state

        best_next = np.max(self.q[nr, nc])

        self.q[r, c, action] += self.alpha * (
            reward + self.gamma * best_next - self.q[r, c, action]
        )

# =====
# SARSA
# =====
class SARSAAgent(BaseAgent):
    def update(self, state, action, reward, next_state, next_action):
        r, c = state
        nr, nc = next_state

        self.q[r, c, action] += self.alpha * (
            reward + self.gamma * self.q[nr, nc, next_action]
            - self.q[r, c, action]
        )

# =====
# Training
# =====
def train_q_learning(env, agent, episodes=500):
    rewards = []

    for ep in range(episodes):
        state = env.reset()
        total_reward = 0

        while True:

```

```

    action = agent.choose_action(state)
    next_state, reward, done = env.step(action)

    agent.update(state, action, reward, next_state)

    state = next_state
    total_reward += reward

    if done:
        break

    rewards.append(total_reward)

return rewards

def run_experiment(runs=50, episodes=500):
    q_all = np.zeros(episodes)
    sarsa_all = np.zeros(episodes)

    for _ in range(runs):
        env = CliffWalkingEnv()

        q_agent = QLearningAgent(4, 12, 4)
        sarsa_agent = SARSAAgent(4, 12, 4)

        q_rewards = train_q_learning(env, q_agent, episodes)
        sarsa_rewards = train_sarsa(env, sarsa_agent, episodes)

        q_all += np.array(q_rewards)
        sarsa_all += np.array(sarsa_rewards)

    return q_all / runs, sarsa_all / runs

def train_sarsa(env, agent, episodes=500):
    rewards = []

    for ep in range(episodes):
        state = env.reset()
        action = agent.choose_action(state)
        total_reward = 0

        while True:
            next_state, reward, done = env.step(action)
            next_action = agent.choose_action(next_state)

            agent.update(state, action, reward, next_state, next_action)

            state = next_state
            action = next_action
            total_reward += reward

        if done:

```

```

        break

    rewards.append(total_reward)

    return rewards

# =====
# Plot Learning Curve
# =====
def plot_learning_curves(q_rewards, sarsa_rewards):
    plt.figure(figsize=(10, 6))

    plt.plot(sarsa_rewards, label='SARSA')
    plt.plot(q_rewards, label='Q-learning')

    plt.xlabel('Episodes')
    plt.ylabel('Reward')
    plt.title('SARSA vs Q-learning (Cliff Walking)')
    plt.legend()
    plt.grid()
    plt.ylim([-200, 0])
    plt.savefig("learning_curve.png")
    plt.show()

# =====
# Policy Extraction
# =====
def get_policy(agent, rows=4, cols=12):
    policy = np.zeros((rows, cols), dtype=int)

    for y in range(rows):
        for x in range(cols):
            policy[y, x] = np.argmax(agent.q[y, x])

    return policy
def extract_path(env, policy, max_steps=100):
    path = []
    state = env.start
    path.append(state)

    for _ in range(max_steps):
        r, c = state
        action = policy[r, c]

        next_state, _, done = env.step(action)
        path.append(next_state)

        if done:
            break

```



```

state = next_state

return path

# =====
# Policy Visualization
# =====
def visualize_policy(policy, title, filename, path=None):
    action_symbols = {0: '↑', 1: '→', 2: '↓', 3: '←'}
    rows, cols = policy.shape

    fig, ax = plt.subplots(figsize=(14, 4))
    ax.set_title(title, fontsize=16)

    ax.set_xlim(0, cols)
    ax.set_ylim(0, rows)
    ax.invert_yaxis()
    ax.axis('off')

    # Grid
    for i in range(rows + 1):
        ax.plot([0, cols], [i, i], color='black')
    for j in range(cols + 1):
        ax.plot([j, j], [0, rows], color='black')

    # 轉成 set (方便查詢)
    path_set = set(path) if path else set()

    for y in range(rows):
        for x in range(cols):

            # Cliff
            if y == rows - 1 and 1 <= x <= cols - 2:
                rect = plt.Rectangle((x, y), 1, 1,
                                     facecolor='lightblue')
                ax.add_patch(rect)

            # Path (綠色)
            elif (y, x) in path_set:
                rect = plt.Rectangle((x, y), 1, 1,
                                     facecolor='lightgreen', alpha=0.6)
                ax.add_patch(rect)

            # Start
            if y == rows - 1 and x == 0:
                ax.text(x + 0.5, y + 0.5,
                       'Start\n↑',
                       ha='center', va='center',
                       fontsize=12, color='blue', fontweight='bold')

```

```
# Goal
elif y == rows - 1 and x == cols - 1:
    ax.text(x + 0.5, y + 0.5,
            'Goal',
            ha='center', va='center',
            fontsize=12, fontweight='bold')

# 一般格子 (箭頭)
elif not (y == rows - 1 and 1 <= x <= cols - 2):
    symbol = action_symbols[policy[y, x]]
    ax.text(x + 0.5, y + 0.5,
            symbol,
            ha='center', va='center',
            fontsize=18, fontweight='bold')

plt.tight_layout()
plt.savefig(filename)
plt.close()

# =====
# Main
# =====
if __name__ == "__main__":

    # ===== 1 多次實驗 (畫 learning curve) =====
    print("Running 50 runs for learning curve...")
    q_rewards, sarsa_rewards = run_experiment()

    plot_learning_curves(q_rewards, sarsa_rewards)

    # ===== 2 單次訓練 (拿 policy) =====
    print("Training single run for policy visualization...")

    env = CliffWalkingEnv()

    q_agent = QLearningAgent(4, 12, 4)
    sarsa_agent = SARSAAgent(4, 12, 4)

    train_q_learning(env, q_agent)
    train_sarsa(env, sarsa_agent)

    # ===== 3 取得 policy =====
    q_policy = get_policy(q_agent)
    sarsa_policy = get_policy(sarsa_agent)

    # ===== 4 產生 path =====
```

```
env.reset()
q_path = extract_path(env, q_policy)

env.reset()
sarsa_path = extract_path(env, sarsa_policy)

# ===== 5 畫圖 =====
visualize_policy(q_policy, "Q-learning Policy", "q_policy.png", q_path)
visualize_policy(sarsa_policy, "SARSA Policy", "sarsa_policy.png", sarsa_path)

print("Done! 已輸出圖片：")
print("- learning_curve.png")
print("- q_policy.png")
print("- sarsa_policy.png")
```

（二）訓練過程

在相同的環境與參數設定下，使用 ϵ -greedy 策略進行訓練，確保兩種方法的公平比較。

（三）結果分析

請針對以下面向進行比較與討論：

學習表現

繪製每一回合的累積獎勵（Total Reward）曲線

比較收斂速度

策略行為

描述或視覺化最終學習到的路徑

分析是否傾向冒險或保守

穩定性分析

比較學習過程中的波動程度

討論探索（exploration）對結果的影響

五、理論比較與討論

在報告中，請說明以下概念：

Q-learning 為離策略（Off-policy）方法，其更新基於「下一狀態的最佳可能行動」，即使該行動未實際執行。

SARSA 為同策略（On-policy）方法，其更新基於「實際採取的行動」，因此會反映探索策略的影響。一般而言：

Q-learning 傾向學習到理論上的最優策略，但在訓練過程中可能較具風險。

SARSA 則傾向學習在實際探索策略下較安全、穩定的行為。

六、結論要求

請總結兩種方法在本實驗中的差異，並說明：

哪一種方法收斂較快

哪一種方法較穩定

在何種情境下應選擇 Q-learning 或 SARSA

修正 readme.md

下面是我幫你整理+修正過的 README.md（可直接貼上用），我主要修了：

- ✗ 混亂的數學表示 → 改成標準 Markdown LaTeX
- ✗ 結構重複與跳躍 → 重新分段
- ✗ SARSA / Q-learning 說明更精準
- ✗ 實驗描述更學術化
- ✗ 強化「作業報告可用版本」

📌 Cliff Walking : Q-learning vs SARSA

一、實驗目的

本實驗旨在比較兩種經典強化學習方法：

- Q-learning (Off-policy)
- SARSA (On-policy)

透過 Cliff Walking 環境分析兩者在以下面向的差異：

- 學習收斂速度
- 策略行為（冒險 vs 保守）
- 穩定性與波動性

二、環境設定 (Cliff Walking)

使用 GridWorld (4×12)：

📌 結構

- 起點 (Start)：左下角
- 終點 (Goal)：右下角
- 懸崖 (Cliff)：底部中間格

🎯 Reward 設計

情況	Reward
每一步	-1
掉入懸崖	-100 (回到起點)
抵達終點	0

三、方法介紹

1 Q-learning (Off-policy)

更新公式：

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

特性：

- 使用 **最大 Q 值** (理想行為)
- 不考慮實際探索行為
- 學習「最優策略」

👉 傾向：

- 更激進
- 可能貼近 cliff 行走

2 SARSA (On-policy)

更新公式：

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma Q(s', a') - Q(s, a) \right]$$

特性：

- 使用「實際選擇的 next action」
- 納入 ϵ -greedy 探索影響
- 學習「實際行為策略」

👉 傾向：

- 更保守
- 遠離 cliff

四、實驗設定

參數	數值
Learning rate (α)	0.1
Discount factor (γ)	0.9
Exploration (ϵ)	0.1
Episodes	500
Runs	50 (取平均)

五、實驗結果

1 學習曲線 (Total Reward)

輸出：

learning_curve.png

觀察結果：

- 初期 reward 很低 (約 -100)
- 隨訓練逐漸改善
- 收斂後：

方法	平均 reward
SARSA	-20 ~ -30
Q-learning	-40 ~ -50

2 策略比較 (Policy)

輸出：

q_policy.png

sarsa_policy.png

Q-learning

- 學習最短路徑
- 貼近 cliff 行走
- 風險較高 (容易掉落)

SARSA

- 避開 cliff
- 路徑較長
- 更穩定安全

3 穩定性分析

方法	特性
Q-learning	波動較大
SARSA	收斂較穩定

原因：

- Q-learning 假設「最佳行為」
- SARSA 考慮 ϵ -greedy 探索噪音

六、重要觀察（重點整理）

1 Q-learning（理論最優）

- 學習 optimal policy
- 但訓練時忽略探索風險
- 行為較危險

2 SARSA（實際策略）

- 學習 behavior policy
- 直接反映探索行為
- 更安全穩定

3 Cliff Walking 的意義

此環境完美展示：

- 👉 Off-policy vs On-policy 差異
- 👉 Risk-aware learning（風險敏感學習）

七、結論

比較結果

項目	Q-learning	SARSA
收斂速度	較快	稍慢
穩定性	較差	較好
策略風險	高（貼 cliff）	低（避開 cliff）

結論

- **Q-learning**
 - 適合：已知環境 or 可安全探索環境
 - 優點：收斂快、學到最優策略
 - 缺點：訓練不穩定
- **SARSA**
 - 適合：高風險環境
 - 優點：穩定、安全
 - 缺點：可能不是理論最優

八、執行方式

Bash

```
python main.py
```

輸出結果：

- learning_curve.png
- q_policy.png
- sarsa_policy.png

如果你要，我也可以幫你再升級成：

- ✓ 教授級報告版本（含圖解說明）
- ✓ 或加上「實驗結果討論段落模板（直接可交）」
- ✓ 或幫你改成 PDF 報告格式



```
import numpy as np
import random
```



```
import matplotlib.pyplot as plt

# =====
# Environment
# =====
class CliffWalkingEnv:
    def __init__(self, rows=4, cols=12):
        self.rows = rows
        self.cols = cols
        self.start = (rows - 1, 0)
        self.goal = (rows - 1, cols - 1)
        self.reset()

    def reset(self):
        self.state = self.start
        return self.state

    def step(self, action):
        r, c = self.state

        # 0: Up, 1: Right, 2: Down, 3: Left
        if action == 0:
            r = max(r - 1, 0)
        elif action == 1:
            c = min(c + 1, self.cols - 1)
        elif action == 2:
            r = min(r + 1, self.rows - 1)
        elif action == 3:
            c = max(c - 1, 0)

        next_state = (r, c)

        # Cliff
        if r == self.rows - 1 and 1 <= c <= self.cols - 2:
            reward = -100
            next_state = self.start
            done = False
        elif next_state == self.goal:
            reward = 0
            done = True
        else:
            reward = -1
            done = False

        self.state = next_state
        return next_state, reward, done

# =====
# Base Agent
# =====
```

```

class BaseAgent:
    def __init__(self, rows, cols, actions,
                  alpha=0.1, gamma=0.9, epsilon=0.1):

        self.q = np.zeros((rows, cols, actions))
        self.alpha = alpha
        self.gamma = gamma
        self.epsilon = epsilon
        self.actions = actions

    def choose_action(self, state):
        if random.random() < self.epsilon:
            return random.randint(0, self.actions - 1)
        else:
            r, c = state
            return np.argmax(self.q[r, c])

# =====
# Q-Learning
# =====
class QLearningAgent(BaseAgent):
    def update(self, state, action, reward, next_state):
        r, c = state
        nr, nc = next_state

        best_next = np.max(self.q[nr, nc])

        self.q[r, c, action] += self.alpha * (
            reward + self.gamma * best_next - self.q[r, c, action]
        )

# =====
# SARSA
# =====
class SARSAAgent(BaseAgent):
    def update(self, state, action, reward, next_state, next_action):
        r, c = state
        nr, nc = next_state

        self.q[r, c, action] += self.alpha * (
            reward + self.gamma * self.q[nr, nc, next_action]
            - self.q[r, c, action]
        )

# =====
# Training
# =====
def train_q_learning(env, agent, episodes=500):

```

```

rewards = []

for ep in range(episodes):
    state = env.reset()
    total_reward = 0

    while True:
        action = agent.choose_action(state)
        next_state, reward, done = env.step(action)

        agent.update(state, action, reward, next_state)

        state = next_state
        total_reward += reward

        if done:
            break

    rewards.append(total_reward)

return rewards

def run_experiment(runs=50, episodes=500):
    q_all = np.zeros(episodes)
    sarsa_all = np.zeros(episodes)

    for _ in range(runs):
        env = CliffWalkingEnv()

        q_agent = QLearningAgent(4, 12, 4)
        sarsa_agent = SARSAAgent(4, 12, 4)

        q_rewards = train_q_learning(env, q_agent, episodes)
        sarsa_rewards = train_sarsa(env, sarsa_agent, episodes)

        q_all += np.array(q_rewards)
        sarsa_all += np.array(sarsa_rewards)

    return q_all / runs, sarsa_all / runs

def train_sarsa(env, agent, episodes=500):
    rewards = []

    for ep in range(episodes):
        state = env.reset()
        action = agent.choose_action(state)
        total_reward = 0

        while True:
            next_state, reward, done = env.step(action)
            next_action = agent.choose_action(next_state)

```

```

agent.update(state, action, reward, next_state, next_action)

state = next_state
action = next_action
total_reward += reward

if done:
    break

rewards.append(total_reward)

return rewards

# =====
# Plot Learning Curve
# =====
def plot_learning_curves(q_rewards, sarsa_rewards):
    plt.figure(figsize=(10, 6))

    plt.plot(sarsa_rewards, label='SARSA')
    plt.plot(q_rewards, label='Q-learning')

    plt.xlabel('Episodes')
    plt.ylabel('Reward')
    plt.title('SARSA vs Q-learning (Cliff Walking)')
    plt.legend()
    plt.grid()
    plt.ylim([-200, 0])
    plt.savefig("learning_curve.png")
    plt.show()

# =====
# Policy Extraction
# =====
def get_policy(agent, rows=4, cols=12):
    policy = np.zeros((rows, cols), dtype=int)

    for y in range(rows):
        for x in range(cols):
            policy[y, x] = np.argmax(agent.q[y, x])

    return policy
def extract_path(env, policy, max_steps=100):
    path = []
    state = env.start
    path.append(state)

    for _ in range(max_steps):
        r, c = state

```

```

    action = policy[r, c]

    next_state, _, done = env.step(action)
    path.append(next_state)

    if done:
        break

    state = next_state

return path

# =====
# Policy Visualization
# =====
def visualize_policy(policy, title, filename, path=None):
    action_symbols = {0: '↑', 1: '→', 2: '↓', 3: '←'}
    rows, cols = policy.shape

    fig, ax = plt.subplots(figsize=(14, 4))
    ax.set_title(title, fontsize=16)

    ax.set_xlim(0, cols)
    ax.set_ylim(0, rows)
    ax.invert_yaxis()
    ax.axis('off')

    # Grid
    for i in range(rows + 1):
        ax.plot([0, cols], [i, i], color='black')
    for j in range(cols + 1):
        ax.plot([j, j], [0, rows], color='black')

    # 轉成 set (方便查詢)
    path_set = set(path) if path else set()

    for y in range(rows):
        for x in range(cols):

            # Cliff
            if y == rows - 1 and 1 <= x <= cols - 2:
                rect = plt.Rectangle((x, y), 1, 1,
                                     facecolor='lightblue')
                ax.add_patch(rect)

            # Path (綠色)
            elif (y, x) in path_set:
                rect = plt.Rectangle((x, y), 1, 1,
                                     facecolor='lightgreen', alpha=0.6)
                ax.add_patch(rect)

```

```

# Start
if y == rows - 1 and x == 0:
    ax.text(x + 0.5, y + 0.5,
            'Start\n↑',
            ha='center', va='center',
            fontsize=12, color='blue', fontweight='bold')

# Goal
elif y == rows - 1 and x == cols - 1:
    ax.text(x + 0.5, y + 0.5,
            'Goal',
            ha='center', va='center',
            fontsize=12, fontweight='bold')

# 一般格子 (箭頭)
elif not (y == rows - 1 and 1 <= x <= cols - 2):
    symbol = action_symbols[policy[y, x]]
    ax.text(x + 0.5, y + 0.5,
            symbol,
            ha='center', va='center',
            fontsize=18, fontweight='bold')

plt.tight_layout()
plt.savefig(filename)
plt.close()

# =====
# Main
# =====
if __name__ == "__main__":

    # ===== 1 多次實驗 (畫 learning curve) =====
    print("Running 50 runs for learning curve...")
    q_rewards, sarsa_rewards = run_experiment()

    plot_learning_curves(q_rewards, sarsa_rewards)

    # ===== 2 單次訓練 (拿 policy) =====
    print("Training single run for policy visualization...")

    env = CliffWalkingEnv()

    q_agent = QLearningAgent(4, 12, 4)
    sarsa_agent = SARSAAgent(4, 12, 4)

    train_q_learning(env, q_agent)
    train_sarsa(env, sarsa_agent)

```

```
# ===== 3 取得 policy =====
q_policy = get_policy(q_agent)
sarsa_policy = get_policy(sarsa_agent)

# ===== 4 產生 path =====
env.reset()
q_path = extract_path(env, q_policy)

env.reset()
sarsa_path = extract_path(env, sarsa_policy)

# ===== 5 畫圖 =====
visualize_policy(q_policy, "Q-learning Policy", "q_policy.png", q_path)
visualize_policy(sarsa_policy, "SARSA Policy", "sarsa_policy.png", sarsa_path)

print("Done! 已輸出圖片：")
print("- learning_curve.png")
print("- q_policy.png")
print("- sarsa_policy.png")
```

(二) 訓練過程

在相同的環境與參數設定下，使用 ϵ -greedy 策略進行訓練，確保兩種方法的公平比較。

(三) 結果分析

請針對以下面向進行比較與討論：

學習表現

繪製每一回合的累積獎勵 (Total Reward) 曲線

比較收斂速度

策略行為

描述或視覺化最終學習到的路徑

分析是否傾向冒險或保守

穩定性分析

比較學習過程中的波動程度

討論探索 (exploration) 對結果的影響

五、理論比較與討論

在報告中，請說明以下概念：

Q-learning 為離策略 (Off-policy) 方法，其更新基於「下一狀態的最佳可能行動」，即使該行動未實際執行。

SARSA 為同策略 (On-policy) 方法，其更新基於「實際採取的行動」，因此會反映探索策略的影響。一般而言：

Q-learning 傾向學習到理論上的最優策略，但在訓練過程中可能較具風險。

SARSA 則傾向學習在實際探索策略下較安全、穩定的行為。

六、結論要求

請總結兩種方法在本實驗中的差異，並說明：

哪一種方法收斂較快

哪一種方法較穩定

在何種情境下應選擇 Q-learning 或 SARSA

根據作業要求撰寫 readme.md 不用加入公式

下面是依照你作業要求整理好的 README.md（已去掉公式、改成報告可直接交版本），結構也幫你對齊老師評分點：

Cliff Walking : Q-learning vs SARSA

一、實驗目的

本實驗比較兩種強化學習方法在 Cliff Walking 環境中的表現：

- Q-learning (Off-policy)
- SARSA (On-policy)

主要分析以下三個面向：

- 學習收斂速度
- 策略行為差異（冒險 vs 保守）
- 學習穩定性

二、實驗環境與設定

Cliff Walking 環境

- Gridworld 大小：4 × 12
- 起點：左下角
- 終點：右下角
- 懸崖區：底部中間區域

Reward 設計

行為	Reward
每一步	-1
掉入懸崖	-100（回到起點）
抵達終點	0

⚙️ 訓練參數

參數	數值
學習率 α	0.1
折扣因子 γ	0.9
探索率 ϵ	0.1
Episodes	500
Runs	50 (取平均)

三、訓練過程

在實驗中：

- Q-learning 與 SARSA 使用相同環境
- 使用 ϵ -greedy 策略
- 保證探索條件一致（公平比較）
- 每回合紀錄 total reward
- 重複 50 次取平均結果

四、結果分析

1 學習表現 (Learning Curve)

輸出圖：

learning_curve.png

📊 觀察結果

- 初期 reward 非常低（頻繁掉入 cliff）
- 隨訓練逐漸改善並收斂
- 最終結果：

方法	表現
SARSA	reward 較高、較穩定

方法	表現
Q-learning	reward 較低、波動較大

🔴 收斂速度

- Q-learning：收斂較快
- SARSA：收斂較慢但穩定

2 策略行為分析 (Policy)

輸出圖：

q_policy.png

sarsa_policy.png

🧠 Q-learning 行為

- 傾向走最短路徑
- 經常貼近 cliff
- 高風險策略（容易掉落）

👉 特性：

- 冒險
- 追求最優解

🧠 SARSA 行為

- 明顯避開 cliff
- 路徑較安全但較長
- 穩定性較高

👉 特性：

- 保守
- 考慮探索風險

3 穩定性分析

方法	穩定性	波動
Q-learning	較差	高

方法	穩定性	波動
SARSA	較好	低

🔪 原因分析

- Q-learning 假設未來永遠採取最佳行為
- SARSA 會考慮 ϵ -greedy 探索帶來的不確定性

👉 因此 SARSA 更貼近「實際行為」

五、理論比較與討論

● Q-learning (Off-policy)

- 學習目標是「理論上的最佳策略」
- 更新時不考慮實際探索行為
- 收斂較快
- 但可能產生危險行為

● SARSA (On-policy)

- 學習「實際執行的策略」
- 更新包含探索行為影響
- 收斂較穩定
- 更安全可靠

🔪 Cliff Walking 的意義

此環境清楚展示：

- Off-policy vs On-policy 差異
- 風險敏感學習 (risk-aware learning)
- 探索策略對結果的影響

六、結論

📊 實驗總結

項目	Q-learning	SARSA
收斂速度	較快	較慢
穩定性	較差	較好
策略風險	高	低
路徑	最短但危險	較安全

📌 最終結論

- 收斂最快：Q-learning
- 最穩定：SARSA

📌 適用情境

Q-learning 適用於：

- 環境可安全探索
- 追求最優策略
- 模擬或理論問題

SARSA 適用於：

- 高風險環境
- 真實世界應用（例如機器人控制）
- 需要穩定行為

七、執行方式

Bash
python main.py

八、輸出結果

執行後會產生：

- learning_curve.png（學習曲線）
- q_policy.png（Q-learning 策略）
- sarsa_policy.png（SARSA 策略）

如果你還想再加分，我可以幫你升級成 🍌：

- ✓ 「教授會給高分的版本（含討論延伸）」
- ✓ 幫你加「實驗誤差分析 + 為什麼Q-learning比較危險」
- ✓ 或直接 幫你改成「PDF報告格式（可交作業）」